NOKIA TETRIS GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive Tetris game using Python Tkinter in which players control falling blocks, arrange them to complete horizontal lines, and earn points. The game provides a graphical interface with score, completed-line and level displays, a preview of the next block, and restart and pause/resume options.

PROCEDURE

Import the required Python libraries, tkinter for the graphical user interface and random for selecting game pieces.

Define the game-board dimensions, cell size, update interval, and the different Tetris block shapes with their colours.

Create the main Tkinter window and set its title to “NOKIA TETRIS”; configure the window background and size.

Create the game-board Canvas and the side panel to display the score, number of completed lines, current level, and next block.

Add the Restart and Pause/Resume buttons to control the game.

Initialize the game board, score, line count, level, and the current and next falling blocks.

Use keyboard controls to move the falling block left or right, move it down, and rotate it where supported by the program.

Update the game at regular intervals so that the current block falls automatically.

Check whether a block can move into the requested position; when it cannot move farther down, fix it to the board and generate the next block.

Check for completed horizontal lines, remove completed lines, shift the remaining blocks downward, and update the score and line count.

Update the level and falling speed according to the game logic, and refresh the next-block preview and game-board display.

Detect when a new block cannot be placed on the board, end the game, and display the game-over state.

Use Restart to begin a new game and Pause/Resume to stop or continue the game; run the program and test the controls, scoring, line clearing, and game-over handling.

PROGRAM CODE:

```
import tkinter as tk
import random

# Nokia-style Tetris game using only Python's built-in Tkinter.
COLS = 10
ROWS = 20
CELL = 28
BOARD_W = COLS * CELL
BOARD_H = ROWS * CELL
TICK_MS = 500

SHAPES = [
    ([[1, 1, 1, 1]], "#38BDF8"),             # I
    ([[1, 1], [1, 1]], "#FACC15"),           # O
    ([[0, 1, 0], [1, 1, 1]], "#A78BFA"),     # T
    ([[0, 1, 1], [1, 1, 0]], "#22C55E"),     # S
    ([[1, 1, 0], [0, 1, 1]], "#EF4444"),     # Z
    ([[1, 0, 0], [1, 1, 1]], "#F97316"),     # J
    ([[0, 0, 1], [1, 1, 1]], "#60A5FA"),     # L
]



class NokiaTetris:
    def __init__(self, root):
        self.root = root
        self.root.title("NOKIA TETRIS")
        self.root.resizable(False, False)
        self.root.configure(bg="#111827")

        tk.Label(
```

```
    root, text="NOKIA TETRIS", font=("Arial Black", 22),
    bg="#111827", fg="#FACC15"
).grid(row=0, column=0, columnspan=2, pady=(12, 6))

self.board_canvas = tk.Canvas(
    root, width=BOARD_W, height=BOARD_H,
    bg="#0F172A", highlightthickness=2, highlightbackground="#38BDF8"
)
self.board_canvas.grid(row=1, column=0, rowspan=2, padx=(14, 10), pady=8)

side = tk.Frame(root, bg="#111827")
side.grid(row=1, column=1, sticky="n", padx=(0, 14), pady=12)

tk.Label(side, text="SCORE", font=("Arial", 10, "bold"),
         bg="#111827", fg="#CBD5E1").pack(anchor="w")
self.score_label = tk.Label(side, text="0", font=("Arial Black", 18),
                            bg="#111827", fg="#FACC15")
self.score_label.pack(anchor="w", pady=(0, 12))

tk.Label(side, text="LINES", font=("Arial", 10, "bold"),
         bg="#111827", fg="#CBD5E1").pack(anchor="w")
self.lines_label = tk.Label(side, text="0", font=("Arial Black", 18),
                            bg="#111827", fg="white")
self.lines_label.pack(anchor="w", pady=(0, 12))

tk.Label(side, text="LEVEL", font=("Arial", 10, "bold"),
         bg="#111827", fg="#CBD5E1").pack(anchor="w")
self.level_label = tk.Label(side, text="1", font=("Arial Black", 18),
                            bg="#111827", fg="#22C55E")
self.level_label.pack(anchor="w", pady=(0, 12))
```

```
    tk.Label(side, text="NEXT", font=("Arial", 10, "bold"),
             bg="#111827", fg="#CBD5E1").pack(anchor="w")
    self.next_canvas = tk.Canvas(side, width=120, height=100, bg="#0F172A",
                                 highlightthickness=1, highlightbackground="#475569")
    self.next_canvas.pack(pady=(4, 12))

    tk.Button(side, text="RESTART", command=self.restart,
              font=("Arial", 10, "bold"), bg="#22C55E", fg="#052E16").pack(fill="x", pady=3)
    tk.Button(side, text="PAUSE / RESUME", command=self.toggle_pause,
              font=("Arial", 9, "bold"), bg="#38BDF8", fg="#082F49").pack(fill="x", pady=3)

    tk.Label(
        root,
        text="← → Move    ↑ Rotate    ↓ Soft drop    SPACE Hard drop    P Pause",
        font=("Arial", 9), bg="#111827", fg="#CBD5E1"
    ).grid(row=3, column=0, columnspan=2, pady=(2, 12))

    self.root.bind("<KeyPress>", self.on_key)
    self.timer = None
    self.restart()

def new_piece(self):
    shape, color = random.choice(SHAPES)
    return {
        "shape": [row[:] for row in shape],
        "color": color,
        "x": COLS // 2 - len(shape[0]) // 2,
        "y": 0
    }

def restart(self):
```

```
    if self.timer is not None:
        try:
            self.root.after_cancel(self.timer)
        except tk.TclError:
            pass
    self.timer = None
    self.board = [[None for _ in range(COLS)] for _ in range(ROWS)]
    self.score = 0
    self.lines = 0
    self.level = 1
    self.paused = False
    self.game_over = False
    self.current = self.new_piece()
    self.next_piece = self.new_piece()
    self.update_labels()
    self.draw()
    self.schedule_tick()

def schedule_tick(self):
    if not self.paused and not self.game_over:
        delay = max(90, TICK_MS - (self.level - 1) * 35)
        self.timer = self.root.after(delay, self.tick)

def toggle_pause(self):
    if self.game_over:
        return
    self.paused = not self.paused
    if self.paused and self.timer is not None:
        try:
            self.root.after_cancel(self.timer)
        except tk.TclError:
```

```
            pass
        self.timer = None
    elif not self.paused:
        self.schedule_tick()
    self.draw()

def on_key(self, event):
    if self.game_over:
        if event.keysym.lower() == "r":
            self.restart()
        return
    key = event.keysym
    if key.lower() == "p":
        self.toggle_pause()
        return
    if self.paused:
        return
    if key == "Left":
        self.move(-1, 0)
    elif key == "Right":
        self.move(1, 0)
    elif key == "Down":
        if self.move(0, 1):
            self.score += 1
            self.update_labels()
    elif key == "Up":
        self.rotate()
    elif key == "space":
        self.hard_drop()
        return
    self.draw()
```

```
def valid(self, piece, dx=0, dy=0, shape=None):
    matrix = shape if shape is not None else piece["shape"]
    for ry, row in enumerate(matrix):
        for rx, filled in enumerate(row):
            if not filled:
                continue
            x = piece["x"] + rx + dx
            y = piece["y"] + ry + dy
            if x < 0 or x >= COLS or y >= ROWS:
                return False
            if y >= 0 and self.board[y][x] is not None:
                return False
    return True

def move(self, dx, dy):
    if self.valid(self.current, dx, dy):
        self.current["x"] += dx
        self.current["y"] += dy
        return True
    if dy == 1:
        self.lock_piece()
    return False

def rotate(self):
    old = self.current["shape"]
    rotated = [list(row) for row in zip(*old[::-1])]
    # Small wall-kick attempts so rotation works near the edges.
    for offset in (0, -1, 1, -2, 2):
        if self.valid(self.current, dx=offset, shape=rotated):
            self.current["x"] += offset
```

```
            self.current["shape"] = rotated
            return

    def hard_drop(self):
        distance = 0
        while self.valid(self.current, dy=1):
            self.current["y"] += 1
            distance += 1
        self.score += distance * 2
        self.lock_piece()
        self.update_labels()
        self.draw()
        self.reset_timer()

    def reset_timer(self):
        if self.timer is not None:
            try:
                self.root.after_cancel(self.timer)
            except tk.TclError:
                pass
            self.timer = None
        self.schedule_tick()

    def tick(self):
        self.timer = None
        if self.paused or self.game_over:
            return
        if not self.move(0, 1):
            if self.game_over:
                self.draw()
                return
```

```
    self.draw()
    self.schedule_tick()

def lock_piece(self):
    piece = self.current
    for ry, row in enumerate(piece["shape"]):
        for rx, filled in enumerate(row):
            if filled:
                x = piece["x"] + rx
                y = piece["y"] + ry
                if y < 0:
                    self.end_game()
                    return
                self.board[y][x] = piece["color"]

    self.clear_lines()
    self.current = self.next_piece
    self.next_piece = self.new_piece()
    if not self.valid(self.current):
        self.end_game()

def clear_lines(self):
    remaining = [row for row in self.board if any(cell is None for cell in row)]
    cleared = ROWS - len(remaining)
    if cleared:
        self.board = [[None for _ in range(COLS)] for _ in range(cleared)] + remaining
        points = {1: 100, 2: 300, 3: 500, 4: 800}
        self.score += points.get(cleared, cleared * 200) * self.level
        self.lines += cleared
        self.level = self.lines // 10 + 1
        self.update_labels()
```

```
def end_game(self):
    self.game_over = True
    self.timer = None

def update_labels(self):
    self.score_label.config(text=str(self.score))
    self.lines_label.config(text=str(self.lines))
    self.level_label.config(text=str(self.level))

def draw_cell(self, canvas, x, y, color, cell=CELL):
    x1, y1 = x * cell, y * cell
    canvas.create_rectangle(x1, y1, x1 + cell, y1 + cell,
                            fill=color or "#0F172A", outline="#1E293B", width=1)
    if color:
        canvas.create_line(x1 + 3, y1 + 3, x1 + cell - 3, y1 + 3,
                           fill="#FFFFFF", width=1)

def draw(self):
    self.board_canvas.delete("all")
    for y in range(ROWS):
        for x in range(COLS):
            self.draw_cell(self.board_canvas, x, y, self.board[y][x])

    if not self.game_over:
        for ry, row in enumerate(self.current["shape"]):
            for rx, filled in enumerate(row):
                if filled:
                    x = self.current["x"] + rx
                    y = self.current["y"] + ry
                    if y >= 0:
```

```
                    self.draw_cell(self.board_canvas, x, y, self.current["color"])

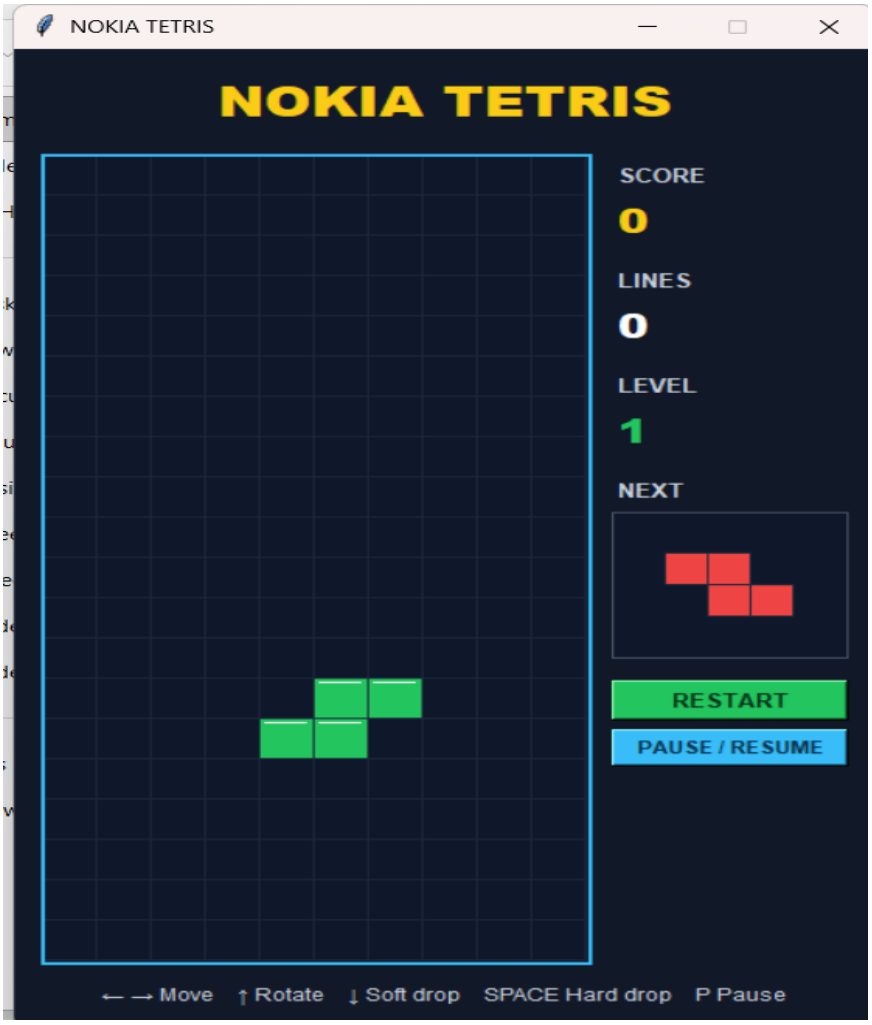
        self.next_canvas.delete("all")
        shape = self.next_piece["shape"]
        cell = 22
        offset_x = (120 - len(shape[0]) * cell) // 2
        offset_y = (100 - len(shape) * cell) // 2
        for ry, row in enumerate(shape):
            for rx, filled in enumerate(row):
                if filled:
                    x1 = offset_x + rx * cell
                    y1 = offset_y + ry * cell
                    self.next_canvas.create_rectangle(
                        x1, y1, x1 + cell, y1 + cell,
                        fill=self.next_piece["color"], outline="#1E293B", width=1
                    )

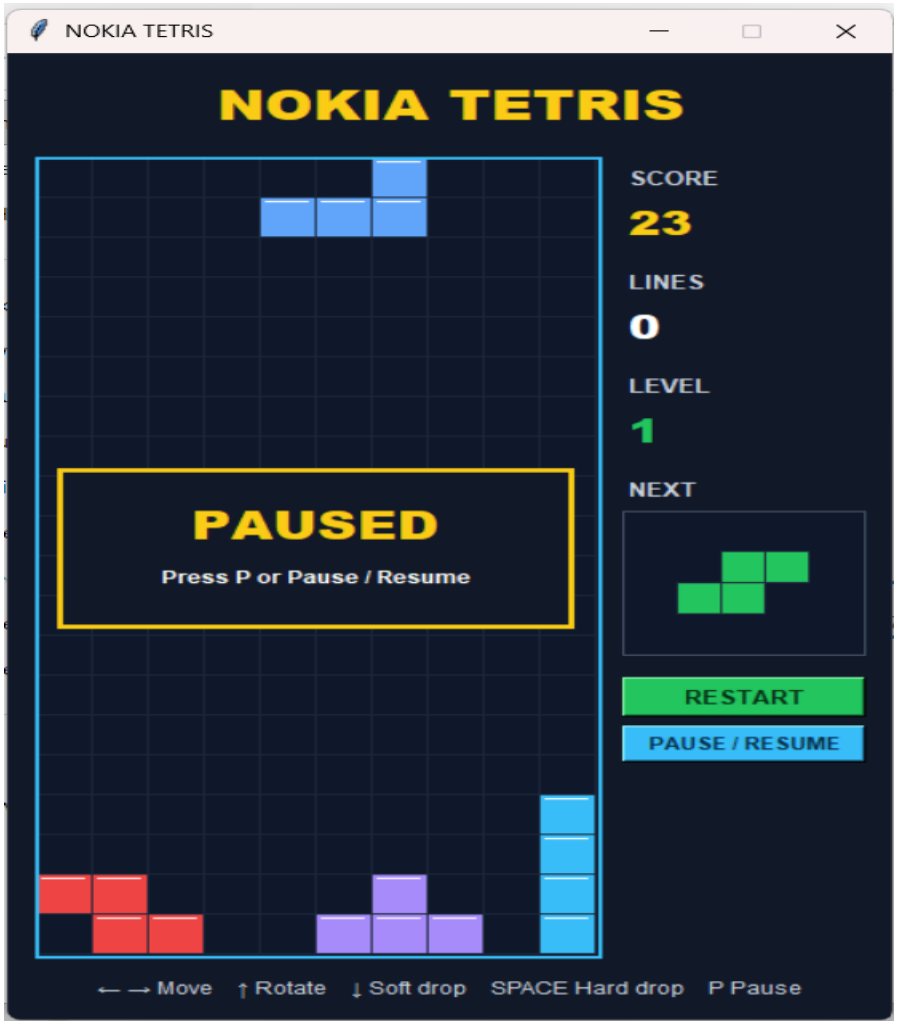
        if self.paused:
            self.draw_overlay("PAUSED", "Press P or Pause / Resume")
        elif self.game_over:
            self.draw_overlay("GAME OVER", "Press RESTART or R to play again")

    def draw_overlay(self, title, subtitle):
        self.board_canvas.create_rectangle(12, 220, BOARD_W - 12, 330,
                                           fill="#111827", outline="#FACC15", width=3)
        self.board_canvas.create_text(BOARD_W // 2, 258, text=title,
                                      font=("Arial Black", 20), fill="#FACC15")
        self.board_canvas.create_text(BOARD_W // 2, 295, text=subtitle,
                                      font=("Arial", 9, "bold"), fill="white")
```

```
if __name__ == "__main__":
    root = tk.Tk()
    game = NokiaTetris(root)
    root.mainloop()
```

OUTPUT:


NOKIA TETRIS
NOKIA TETRIS
SCORE
0
LINES
0
LEVEL
1
NEXT
RESTART
PAUSE / RESUME
← → Move ↑ Rotate ↓ Soft drop SPACE Hard drop P Pause



RESULT

Thus, a Nokia-style Tetris game was developed using Python Tkinter. The game provides a graphical board, falling and controllable blocks, line-clearing and score updates, level information, a next-block preview, and restart and pause/resume controls. The program demonstrates GUI design, keyboard event handling, random selection, collision checking, and timer-based game updates.